

IMAGE PROCESSING · STUDY GUIDE

Chapter 9

Morphological Image Processing

A Complete Ground-Up Reference

From the mathematics of sets to pixel-by-pixel grid operations and advanced morphological algorithms. Includes worked examples, visual grids, and practice questions.

Part 1 · Set Theory

Part 2 · Binary Images

Part 3 · Structuring Elements

Part 4 · Erosion

Part 5 · Dilation

Part 6 · Opening & Closing

Part 7 · Algorithms

Part 8 · Practice

Table of Contents

Part 1 · Mathematical Foundations (Set Theory)	3
1.1 Elementary Set Operations	3
1.2 Geometric Set Operations	4
Worked Problem 1.1 — Geometric Transformations	4
Part 2 · Representing Binary Images as Coordinate Sets	5
Worked Problem 2.1 — Mapping a Grid to a Set	5
Part 3 · The Structuring Element (SE)	6
Part 4 · Erosion ($\ominus$)	7
Worked Problem 4.1 — Erosion in 2D	7
Part 5 · Dilation ($\oplus$)	9
Worked Problem 5.1 — Dilation in 2D	9
Part 6 · Opening ($\circ$) & Closing ($\bullet$)	11
6.1 Opening	11
6.2 Closing	11
Worked Problem 6.1 — Opening vs. Closing (1D)	11
Part 7 · Morphological Algorithms & Applications	12
7.1 Boundary Extraction $\beta(A)$	12
7.2 Hole Filling	13
Worked Problem 7.1 — Hole Filling	13
7.3 Connected Component Extraction	15
Part 8 · Practice Questions	16

PART 1

Mathematical Foundations (Set Theory)

In mathematical morphology, binary images are treated as **sets of pixel coordinates**. A pixel value of **1** (foreground) belongs to set A ; a pixel value of **0** (background) does not. Every pixel at row r , column c is written as the coordinate pair (r, c) .

1.1 Elementary Set Operations

Let A and B be sets of coordinate points in a 2D integer space $\mathbb{Z}^2$.

Union ($\cup$)

Combines all coordinate points from both sets — behaves like a **logical OR**.

$$C = A \cup B = \{ w \mid w \in A \text{ or } w \in B \}$$

Intersection ($\cap$)

Contains only the coordinate points present in *both* sets — behaves like a **logical AND**.

$$D = A \cap B = \{ w \mid w \in A \text{ and } w \in B \}$$

Complement (A^c)

All coordinate points in our space that do *not* belong to A — behaves like a **logical NOT**.

$$A^c = \{ w \mid w \notin A \}$$

Set Difference ($A - B$)

Removes any elements of B from set A .

$$A - B = \{ w \mid w \in A \text{ and } w \notin B \} = A \cap B^c$$

1.2 Geometric Set Operations

These operations physically shift or mirror coordinates in 2D space and form the core mechanics of morphological filtering.

Reflection ($\hat{B}$)

Flips a set B of coordinate offsets about its origin $(0, 0)$.

$$\hat{B} = \{ w \mid w = -b, \text{ for } b \in B \}$$

Translation $((A)_z)$

Shifts every coordinate point in A by displacement vector $z = (z_x, z_y)$.

$$(A)_z = \{ w \mid w = a + z, \text{ for } a \in A \}$$

WORKED PROBLEM 1.1 — GEOMETRIC TRANSFORMATIONS

Let set B be given by three coordinate offsets relative to origin $(0,0)$:

$$B = \{ (0,0), (0,1), (1,0) \}$$

1. Reflection ($\hat{B}$)

Multiply each coordinate by -1 :

$$\hat{B} = \{ (-0,-0), (-0,-1), (-1,-0) \} = \{ (0,0), (0,-1), (-1,0) \}$$

2. Translation of B by $z = (2, 3)$

Add $(2, 3)$ to each coordinate pair:

$$(B)_z = \{ (0+2, 0+3), (0+2, 1+3), (1+2, 0+3) \} = \{ (2,3), (2,4), (3,3) \}$$

PART 2

Representing Binary Images as Coordinate Sets

To process a binary grid morphologically, we map its **1**-valued pixels into a coordinate set. We use the following standard coordinate system:

- Top-left corner is **(0, 0)**
- Row index increases **downward**
- Column index increases **rightward**

Cols (x) →
 (0, 0) (0, 1) (0, 2)
 (1, 0) (1, 1) (1, 2)
 (2, 0) (2, 1) (2, 2)

WORKED PROBLEM 2.1 — MAPPING A GRID TO A SET

Translate this 3×3 binary matrix into a coordinate set A:

Locate every cell containing **1**:

0	1	0
1	1	0
0	0	0

Matrix M

- Row 0, Col 1 → (0, 1)
- Row 1, Col 0 → (1, 0)
- Row 1, Col 1 → (1, 1)

$A = \{ (0,1), (1,0), (1,1) \}$

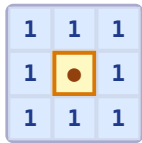
PART 3

The Structuring Element (SE)

A **Structuring Element (SE)** is a small binary template (commonly 3×3) used to probe an image. When specifying an SE, you must define:

1. **The Shape** — e.g., cross, square, horizontal line.
2. **The Origin** — the reference center, typically marked  or specified as (0, 0).

Example 1 — Symmetric 3×3 Square (B_1)



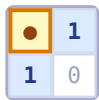
Origin: Center pixel (0, 0)

Offsets: $B_1 = \{ (-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 0), (0, 1), (1, -1), (1, 0), (1, 1) \}$

Note: Being symmetric, its reflection is identical — $\hat{B}_1 = B_1$

B_1 — Square SE

Example 2 — Asymmetric L-Shape (B_2)



Origin: Top-left pixel (0, 0)

Offsets: $B_2 = \{ (0, 0), (0, 1), (1, 0) \}$

Reflection: $\hat{B}_2 = \{ (0, 0), (0, -1), (-1, 0) \}$

B_2 — L-Shape SE

PART 4

Erosion ($\ominus$)

Erosion is a *contraction* operator. It shrinks foreground structures, removes small noise components, and breaks thin connections between regions.

MATHEMATICAL DEFINITION

The erosion of image A by structuring element B is the set of all displacement points z such that B , when translated by z , fits *entirely inside* A :

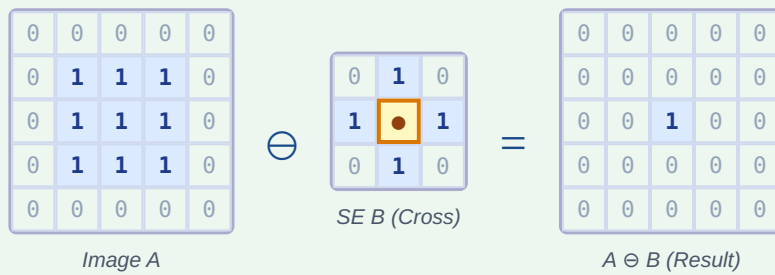
$$A \ominus B = \{ z \mid (B)_z \subseteq A \}$$

INTUITIVE GRID RULE

Place the origin of the SE over a pixel. If **every single 1** in the SE aligns with a **1** in the image → output is **1**. If even one 1 in the SE overlaps a 0 → output is **0**.

WORKED PROBLEM 4.1 — EROSION IN 2D

Given image A (5×5) and a cross-shaped structuring element B (origin at center):



We evaluate the central region (rows 1–3, cols 1–3, 0-indexed). Boundary pixels are copied as 0.

1 At position (1,1) [value = 1]

The neighborhood around (1,1) is:

0	0	0
0	1	1
0	1	1

Neighborhood

The cross SE requires a 1 at its top position (0,1), but the image has 0 there.

→ **FAILS**. Output at (1,1) = 0

2 At position (2,2) [value = 1] — the center

The neighborhood around (2,2) is all 1s:

1	1	1
1	1	1
1	1	1

Neighborhood

Every 1 in B aligns with a 1 in A . The SE is a subset.

→ **PASSES**. Output at (2,2) = 1

All other interior positions fail because the SE extends into the background border of 0s. Only the single center pixel survives.

PART 5

Dilation ($\oplus$)

Dilation is an *expansion* operator. It grows foreground structures, fills small holes, and connects close, separated components.

MATHEMATICAL DEFINITION

The dilation of A by B is the set of all displacements z such that the reflected structuring element $\hat{B}$, translated by z , overlaps with A at least one element:

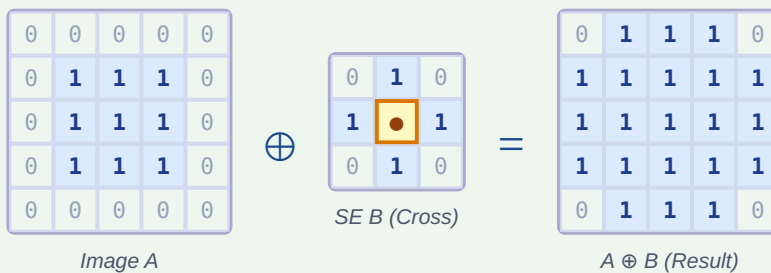
$$A \oplus B = \{ z \mid (\hat{B})_z \cap A \neq \emptyset \}$$

INTUITIVE GRID RULE

Place the origin of the SE over a pixel. If **at least one 1** in the SE overlaps a **1** in the image $\rightarrow$ output is **1**. If the SE lies entirely over 0s $\rightarrow$ output is **0**.

 WORKED PROBLEM 5.1 — DILATION IN 2D

Using the same image A and cross-shaped SE B from Problem 4.1:


1 At pixel (0,2) [value = 0]

Placing the center of B over (0,2), the bottom row of the SE aligns with pixels (1,1), (1,2), (1,3) — all 1.

$\rightarrow$ **Overlap exists.** Output at (0,2) = 1

2 At pixel (1,0) [value = 0]

The rightmost arm of the SE (pointing right from origin) aligns with the 1 at (1,1).

$\rightarrow$ **Overlap exists.** Output at (1,0) = 1

Applying this check across the entire grid produces the cross-shaped expansion shown above — the object grows by one pixel outward in each cardinal direction.

PART 6

Opening (◦) & Closing (•)

Single passes of erosion or dilation change object sizes. To filter noise *without* altering overall object size, we chain them together.

6.1 Opening (◦)

Opening is an **erosion followed by a dilation**:

$$A \circ B = (A \ominus B) \oplus B$$

KEY CHARACTERISTICS

- Removes small, isolated noise components (white dots on black background)
- Breaks thin, thread-like connections between larger objects
- Smooths the *outer* contours of an object
- **Size Preservation:** Objects larger than the SE survive with boundaries largely intact

6.2 Closing (•)

Closing is a **dilation followed by an erosion**:

$$A \bullet B = (A \oplus B) \ominus B$$

KEY CHARACTERISTICS

- Fills small, dark holes (black dots on a white object)
- Fuses close, disjoint objects together
- Smooths the *inner* contours of an object

WORKED PROBLEM 6.1 — OPENING VS. CLOSING (1D)

Given a noisy 1D binary signal A and a 1D structuring element B (bold = origin):

$A =$



Indices: 0 1 2 3 4 5 6 7 8 9 10 11

$B = [1, \mathbf{1}, 1]$ (bold = origin, window size 3)

Step 1 — Erosion ($A \ominus B$)

A position yields 1 only if all three values under the window are 1:

- Index 2: $[0, 1, 0] \rightarrow 0$ (isolated noise pixel, removed ✓)
- Index 4: $[1, 1, 1] \rightarrow 1$ (indices 4,5,6 all 1)
- Index 5: $[1, 1, 1] \rightarrow 1$ (indices 5,6,7 all 1)
- Index 9: $[0, 1, 0] \rightarrow 0$ (isolated noise pixel, removed ✓)
- All others: contain at least one 0 $\rightarrow 0$

$A \ominus B =$



↑ Noise at indices 2 and 9 successfully removed.

Step 2 — Dilation of eroded result ($(A \ominus B) \oplus B$)

Each 1 expands to include its left and right neighbours:

- 1 at index 4 expands to indices 3, 4, 5
- 1 at index 5 expands to indices 4, 5, 6

$A \circ B =$



↑ Main object restored to its original shape; noise eliminated.

PART 7

Morphological Algorithms & Applications

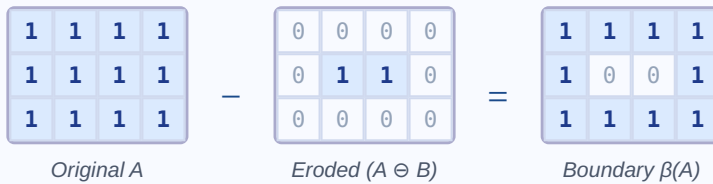
7.1 Boundary Extraction $\beta(A)$

Extracts the **outer edge** of an object by subtracting the eroded image from the original:

$$\beta(A) = A - (A \ominus B)$$

Eroding strips away the outermost layer of pixels. Subtracting the eroded version from the original leaves only the boundary pixels.

VISUAL ILLUSTRATION



7.2 Hole Filling

A "hole" is a background region completely enclosed by foreground pixels. This iterative algorithm fills those internal spaces.

MATHEMATICAL FORMULATION

$$X_k = (X_{k-1} \oplus B) \cap A^c, \quad k = 1, 2, 3, \dots$$

- A — input image containing closed boundaries
- A^c — complement of A ; acts as a mask preventing dilation from leaking outside
- B — symmetric cross-shaped SE
- X_0 — all zeros, except a single 1 placed *inside* the target hole
- **Terminate** when $X_k = X_{k-1}$. Final output: $A \cup X_k$

 WORKED PROBLEM 7.1 — HOLE FILLING

Image A contains a 3×3 hole (the bold 0 at (2,2) marks our seed):

1	1	1	1	1
1	0	0	0	1
1	0	0	0	1
1	0	0	0	1
1	1	1	1	1

Image A (highlighted cell = seed)

The ring of 1s forms a closed boundary around a 3×3 region of 0s. We want to fill the interior.

1 Compute Complement A^c

0	0	0	0	0
0	1	1	1	0
0	1	1	1	0
0	1	1	1	0
0	0	0	0	0

A^c — complement

2 Initialize X_0 — single 1 at the seed position (2,2):

0	0	0	0	0
0	0	0	0	0
0	0	1	0	0
0	0	0	0	0
0	0	0	0	0

X_0

3 Iteration 1: $X_1 = (X_0 \oplus B) \cap A^c$

0	0	0	0	0
0	0	1	0	0
0	1	1	1	0
0	0	1	0	0
0	0	0	0	0

$X_0 \oplus B$

$\cap$

0	0	0	0	0
0	1	1	1	0
0	1	1	1	0
0	1	1	1	0
0	0	0	0	0

A^c

$=$

0	0	0	0	0
0	0	1	0	0
0	1	1	1	0
0	0	1	0	0
0	0	0	0	0

X_1

4 Iteration 2: $X_2 = (X_1 \oplus B) \cap A^c$

0	0	1	0	0
0	1	1	1	0
1	1	1	1	1
0	1	1	1	0
0	0	1	0	0

 $\cap A^c =$

0	0	0	0	0
0	1	1	1	0
0	1	1	1	0
0	1	1	1	0
0	0	0	0	0

$X_1 \oplus B$
 X_2 (hole filled!)

Intersection with A^c clips expansion at the boundaries — growth stays inside the hole.

Termination: $X_3 = X_2$, so the algorithm stops. Final output = $A \cup X_2$:

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

Filled Image = $A \cup X_2$

7.3 Connected Component Extraction

Isolates a specific, continuous structure from an image containing multiple objects using the same iterative dilation strategy:

MATHEMATICAL FORMULATION

$$X_k = (X_{k-1} \oplus B) \cap A, \quad k = 1, 2, 3, \dots$$

- **A** — input image with multiple objects
- **B** — 8-connected square SE (all ones in 3×3)
- **X₀** — single 1 placed on a known pixel of the *target* component
- **Terminate** when $X_k = X_{k-1}$ — only the target component is isolated

Note: Unlike Hole Filling (which uses A^c as a mask), this algorithm intersects with *A itself* — constraining growth to foreground pixels only.

PART 8

Practice Questions

Test your understanding before the exam. Attempt each question fully before checking the hints.

QUESTION 1 — COORDINATE CONVERSIONS

Given a 4×4 binary image matrix M :

0	1	0	0
0	1	1	0
0	0	0	0
1	0	0	1

Matrix M

1. Translate this matrix into a coordinate set A .
2. Find the translation of set A by vector $z = (1, -1)$.

HINT

1. $A = \{ (0,1), (1,1), (1,2), (3,0), (3,3) \}$
2. $(A)_z = \{ (1,0), (2,0), (2,1), (4,-1), (4,2) \}$

QUESTION 2 — EROSION AND DILATION ON A 2D GRID

Given image A and horizontal line-shaped SE B (bold = center origin):

0	0	0	0
0	1	1	0
0	1	1	0
0	0	0	0

Image A

$$\text{SE } B = [\mathbf{1} \bullet \mathbf{1}]$$

Horizontal, window size 3

1. Compute the erosion $A \ominus B$.
2. Compute the dilation $A \oplus B$.

HINT

1. The line SE requires a 3-pixel-wide horizontal block of 1s. Since the 1-region is only 2 pixels wide horizontally, the entire eroded result is **all zeros**.
2. For dilation, the shape expands horizontally by 1 pixel on each side (left and right).

QUESTION 3 — BOUNDARY EXTRACTION

Using the same image A and structuring element B from Question 2:

1. Compute the boundary extraction $\beta(A) = A - (A \ominus B)$.
2. Explain how using a *horizontal line* SE affects the extracted boundary compared to a 3×3 square SE.

HINT

1. Since $A \ominus B = 0$ (all zeros), the boundary extraction $\beta(A) = A - 0 = \mathbf{A \text{ itself}}$.
2. A horizontal line element only strips horizontal neighbours, leaving vertical edges intact — so the result shows more of the interior structure compared to a square SE, which strips in all directions.

QUESTION 4 — CONCEPTUAL ANALYSIS

1. Why does the dilation formula require the *reflection* of the SE ($\hat{B}$) before translation, while erosion does not?
2. Under what condition is the reflection step in dilation mathematically redundant?

HINT

1. Dilation is the union of all translations of the image by the active points of the SE — structurally equivalent to a spatial convolution, which requires reflection to preserve directionality.
2. The reflection step is redundant when the structuring element is **symmetric about its origin**, i.e., when $\hat{B} = B$ (e.g., the cross or square SE).